COER UNIVERSITY

7th km, Roorkee-Haridwar Highway, Vardhmanpuram, Roorkee, Uttarakhand

Lab Sheet-08: Mini Projects on Deep Learning and Reinforcement Learning

MCA III Semester (Session 2026–2027)

Project 1: Handwritten Digit Recognition using Neural Networks

Objective: Develop a Deep Neural Network (DNN) using the MNIST dataset to recognize handwritten digits and evaluate model accuracy.

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np

# 1. Load and preprocess the MNIST dataset
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize pixel values to be between 0 and 1
x_train, x_test = x_train / 255.0, x_test / 255.0

# 2. Build the Deep Neural Network (DNN) Architecture
model = models.Sequential([
    layers.Flatten(input_shape=(28, 28)),    # Flatten 2D image matrix to 1D vector
    layers.Dense(128, activation='relu'),    # Hidden layer with ReLU activation
    layers.Dropout(0.2),                     # Regularization layer to prevent overfitting
    layers.Dense(10, activation='softmax')   # Output layer for 10 classes (digits 0-9)
])

# 3. Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# 4. Train the model and save the history for learning curves
print("--- Training the DNN Model ---")
history = model.fit(x_train, y_train, epochs=5, validation_split=0.1, batch_size=64)

# 5. Evaluate the model on test data
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"\nTest Accuracy: {test_acc:.4f}")

# 6. Plot Learning Curves
```

```
plt.figure(figsize=(12, 4))

# Plot Accuracy curve
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Plot Loss curve
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.savefig('mnist_learning_curves.png')
print("Learning curves saved as 'mnist_learning_curves.png'")

# 7. Save the trained model
model.save('mnist_dnn_model.h5')
print("Model saved successfully as 'mnist_dnn_model.h5'")
```

Project 2: Q-Learning based Maze Solver

Objective: Develop a Reinforcement Learning agent that learns the shortest path in a maze using the Q-Learning algorithm and reward optimization.

```
import numpy as np

# 1. Environment Setup (4x4 Maze Grid)
#  0: Safe/Empty cell
# -1: Wall/Obstacle
# 10: Goal/Target
maze = np.array([
    [0,   0,  0, -1],
    [-1, -1,  0, -1],
    [0,   0,  0,  0],
    [-1,  0, -1, 10]
])

# Hyperparameters
alpha = 0.1      # Learning rate
gamma = 0.9      # Discount factor
epsilon = 0.2    # Exploration rate
episodes = 500   # Total training episodes

# Q-table Initialization (16 states x 4 actions)
# Actions: 0=Up, 1=Down, 2=Left, 3=Right
q_table = np.zeros((16, 4))
```

```
def get_state(row, col):
    return row * 4 + col

def move(row, col, action):
    orig_row, orig_col = row, col
    if action == 0 and row > 0: row -= 1     # Up
    elif action == 1 and row < 3: row += 1   # Down
    elif action == 2 and col > 0: col -= 1   # Left
    elif action == 3 and col < 3: col += 1   # Right

    # Hit a wall -> remain in the original position
    if maze[row, col] == -1:
        return orig_row, orig_col
    return row, col

print("--- Training the Reinforcement Learning Agent ---")
for ep in range(episodes):
    row, col = 0, 0  # Starting state (0,0)

    while maze[row, col] != 10:
        state = get_state(row, col)

        # Epsilon-greedy action selection
        if np.random.uniform(0, 1) < epsilon:
            action = np.random.choice(4)
        else:
            action = np.argmax(q_table[state])

        next_row, next_col = move(row, col, action)
        next_state = get_state(next_row, next_col)

        # Reward formulation
        if maze[next_row, next_col] == 10:
            reward = 10
        else:
            reward = -1  # Step penalty to encourage shortest path

        # Bellman Optimality Equation (Q-table update)
        best_next_q = np.max(q_table[next_state])
        q_table[state, action] += alpha * (reward + gamma * best_next_q - q_table[state, action]

        row, col = next_row, next_col

print("Training completed.")

# 2. Test the optimal path found by the agent
print("\n--- Testing Optimal Maze Path ---")
row, col = 0, 0
path = [(row, col)]
steps = 0

while maze[row, col] != 10 and steps < 20:
    state = get_state(row, col)
    action = np.argmax(q_table[state])
    row, col = move(row, col, action)
    path.append((row, col))
    steps += 1
```

```
print(f"Optimal Path coordinates: {path}")
```